UTM

UNIVERSITI TEKNOLOGI MALAYSIA

GROUP PROJECT

SECR2043

SECTION 03

OPERATING SYSTEM

Title : Virtual Memory Management – Replacement Algorithm: OPT, LRU

LECTURER: DR. DANLAMI GABI

NO.	NAME	MATRIC NUMBER
1.		
2.	AFIF SHAQIR IRFAN BIN ARQAM	A23CS0204
3.	ANIS SAFIYYA BINTI JANAI	A23CS0049
4.	NURUL ASYIKIN BINTI KHAIRUL ANUAR	A23CS0162

Virtual Memory Management – Replacement Algorithm: OPT

1.0 Title & Abstract

Modern operating systems use virtual memory to handle processes efficiently, with page replacement algorithms playing a critical role in memory management. In real-world systems such as video streaming platforms, similar memory challenges occur in caching strategies, especially at the edge server level. By leveraging predictive analytics, such platforms can use advanced algorithms like OPT to prefetch and retain video segments that are more likely to be accessed in the near future.

2.0 Introduction

Modern operating systems use virtual memory to handle processes efficiently, with page replacement algorithms playing a critical role in memory management. In real-world systems such as video streaming platforms, similar memory challenges occur in caching strategies, especially at the edge server level. By leveraging predictive analytics, such platforms can use advanced algorithms like OPT to prefetch and retain video segments that are more likely to be accessed in the near future.

3.0 Problem Statement

In video streaming systems, cache servers store frequently accessed video segments to reduce loading times. However, due to limited memory space, not all requested segments can be retained. The challenge lies in identifying which segments to evict when the cache is full. Traditional algorithms like FIFO and LRU often make suboptimal decisions in this context. This project addresses this challenge by implementing the Optimal Page Replacement Algorithm, which offers the lowest possible page fault rate by utilizing future access patterns.

4.0 Related Works

Numerous studies have explored memory replacement strategies in operating systems.

- Belady's Algorithm (1966) introduced the theoretical foundation of the OPT algorithm, showing that knowing future page requests minimizes page faults.

- LRU and FIFO are widely implemented due to their simplicity, but they fail in scenarios where predictive analytics can be leveraged.

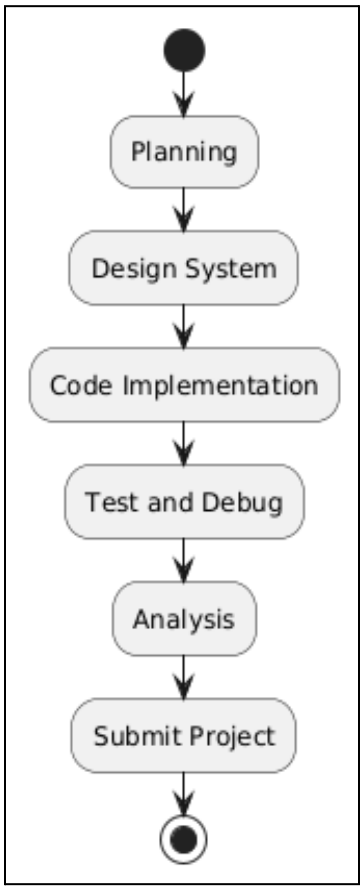
- In cloud computing and content delivery networks (CDNs), modern streaming platforms have started integrating AI-driven cache management systems, similar in spirit to OPT, using user behavior predictions.

This project builds on these concepts to create a simplified simulation of such a system.

5.0 Methodology

This project utilizes a structured development strategy to simulate the OPT replacement algorithm. The methodology consists of the project management flowchart with the hardware and software requirements to ensure the simulations can run smoothly.

5.1 Project Management Flowchart

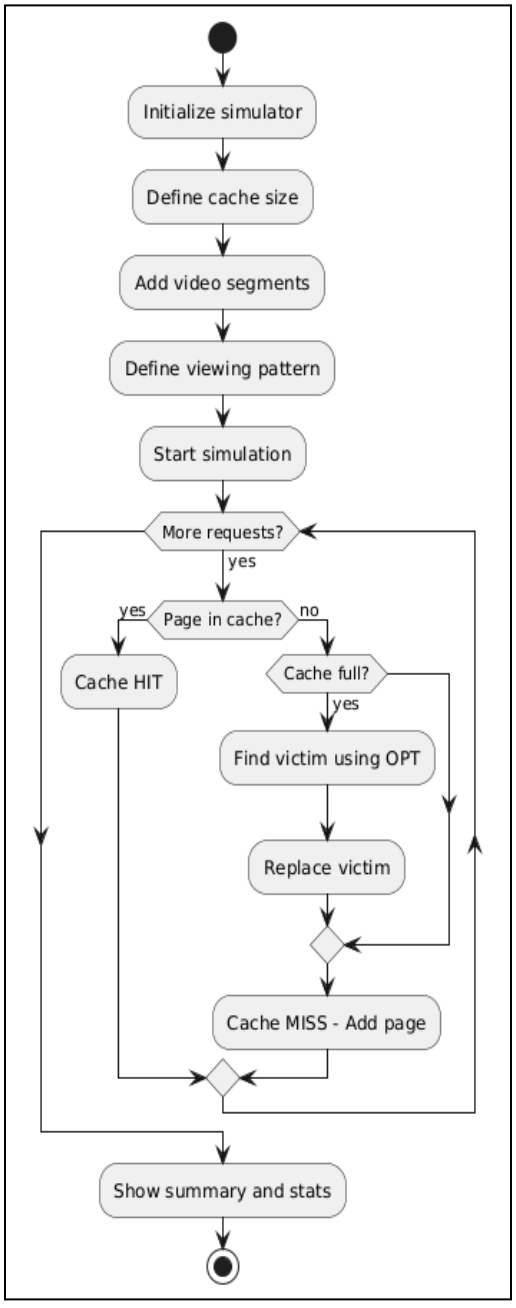


1. Planning - Define project objectives, scope, timeline.
2. Design System - Design the system architecture
3. Code Implementation - Implement core logic and features,
4. Test and Debug - Run and Identify bugs to fix
5. Analysis the system - Review the simulation
6. Submit the project - Finalize the documents

5.2 Hardware and Software Requirements

- **Software**: Google Colab for discrete-event simulation, Python for coding.
- **Hardware**: Standard computational resources for running simulations

6.0 Design and Implementation



6.1 System Design

Based on the flowchart above, the system design flow is as stated below :-

1. Start

2. Initialize simulator - The system is prepared with a cache of fixed size using the VideoStreamingCacheSimulator class.

3. Define cache size - 400mb for example

4. Add video segments - Each video segment (e.g., goal highlights, replays) is registered with metadata

5. Define viewing pattern (will be fed into the OPT) - A list of segment IDs simulating user viewing behavior is created

6. Start simulation - The system begins processing requests using the **Optimal Page Replacement (OPT)** algorithm

7. Stop

6.2 Code Implementation

Optimal Page Replacement

For the simulation of the code, we decided to use python script to implement the Optimal Page Replacement Algorithm. Inside the code, there are several main components which is the VideoStreamingCacheSimulator: It's the brain behind the cache management. The VideoStreamingCacheSimulator feeds it the sequence of video requests, and OPTPageReplacement decides which segments to keep and which to remove to maintain peak efficiency, showcasing the best-case scenario for video content caching.

Initialization of class Video Streaming Cache Simulator

```
class VideoStreamingCacheSimulator:

    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, cache_size):

        self.cache_size = cache_size
        self.opt_algorithm = OPTPageReplacement(cache_size)
        self.video_segments = {}  # Maps segment IDs to video info
```

Based on the figure, this is the class in which we will build scenario that surrounding the OPT Page Replacement which contained:

- self.cache_size: This parameter defines the maximum number of video segments that the edge server's cache can hold. This is analogous to frame_size in the generic OPTPageReplacement class.
- self.opt_algorithm = OPTPageReplacement(cache_size): This line is crucial as it instantiates an object of the OPTPageReplacement class, passing the cache_size. This means that all the logic for determining hits, faults, and, most importantly, finding the optimal victim for replacement will be handled by this embedded opt_algorithm instance.

- self.video_segments = {}: This is an empty dictionary initialized to store metadata about each video segment. It will map segment_id (the page number in OPTPageReplacement terms) to a dictionary containing video_title and segment_info. This allows the simulator to provide richer, video-specific descriptions in its output.

Defining the scenario - Sports Live Streaming Video

For the scenario itself which is a method/function, we will demonstrate the *Sports Live Streaming Video* which is a High-demand sports event with predictable replay patterns that will invoke the demonstration of the page replacement as visualized below:

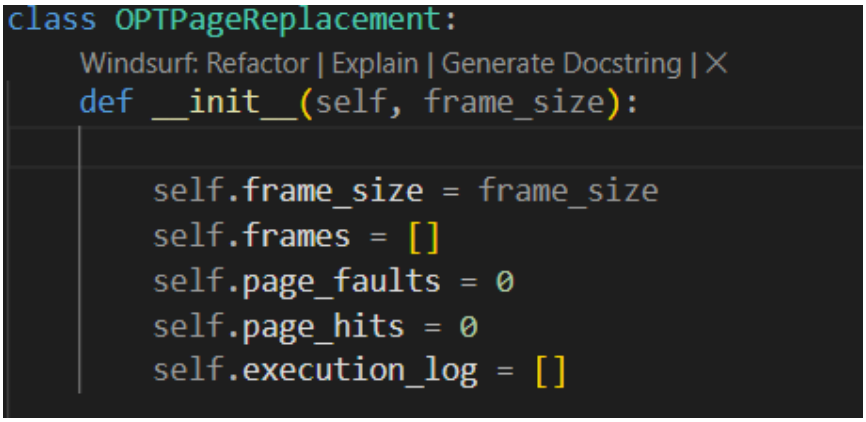
```
def sports_live_streaming_scenario():

    print("\n" + "="*80 + "\n")
    print("⚽ SCENARIO 2: LIVE SPORTS STREAMING")
    print("Event: FIFA World Cup Final - High replay demand")
    print()

    # Initialize cache for 3 video segments (smaller cache, higher demand)
    cache_sim = VideoStreamingCacheSimulator(cache_size=3)

    # Add sports segment information
    cache_sim.add_video_segment(10, "World Cup Final", "Live Stream - Main Feed")
    cache_sim.add_video_segment(11, "Goal Highlight 1", "Messi's Goal - 23rd minute")
    cache_sim.add_video_segment(12, "Goal Highlight 2", "Mbappe's Goal - 45th minute")
    cache_sim.add_video_segment(13, "Penalty Shootout", "Final Penalty Sequence")
    cache_sim.add_video_segment(14, "Victory Celebration", "Team Celebration")
    cache_sim.add_video_segment(15, "Post-Match Interview", "Player Interviews")

    # Viewing pattern: Live stream + frequent replays of key moments
    viewing_pattern = [10, 11, 12, 10, 11, 13, 14, 11, 12, 13, 15, 11]

    results = cache_sim.simulate_viewing_requests(
        viewing_pattern,
        "Live Sports Event - High Replay Demand"
    )

    return results
```

Firstly, a VideoStreamingCacheSimulator object is instantiated. Crucially, the cache_size is set to 3. This represents a relatively smaller cache, highlighting that live sports events often have a very dynamic and concentrated set of highly demanded segments (e.g., live feed, recent highlights), requiring efficient use of limited cache space.

Then, several cache_sim.add_video_segement() calls are made to register different video segment that might be requested during a live sports event that include *Live Stream - Main Feed, Messi's Goal - 23rd minute, Mbappe's Goal - 45th minute, Final Penalty Sequence, Team Celebration, and Player Interviews.*

After that, the viewing_pattern = [10, 11, 12, 10, 11, 13, 14, 11, 12, 13, 15, 11] This list is the core input for the OPT algorithm's simulation. It represents the sequence of requests for video segments

Initialization of class OPT Page Replacement



The figure above is the class that simulates the Optimal Page Replacement algorithm, which is a theoretical ideal for memory management. In the context of the VideoStreamingCacheSimulator, it acts as the intelligent caching engine.

The way this class works is When the cache is full and a new video segment is requested, OPTPageReplacement uses perfect future knowledge (provided by the *viewing_pattern* in the simulation) to look ahead. It identifies and evicts the video segment currently in the cache that will not be needed for the longest time in the future, or never again. This "clairvoyance" ensures that the most useful segments remain cached, leading to the absolute fewest possible cache misses.

Execution of the code

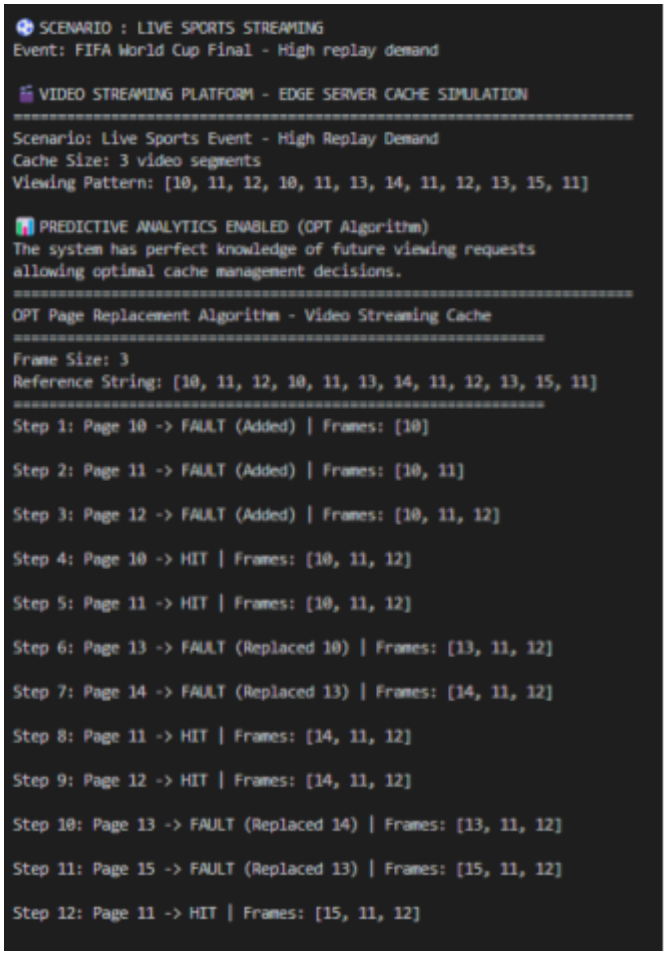
```python
# Main execution
if __name__ == "__main__":

    # Run all scenarios
    scenario_results = []

    # Scenario : Sports Live Streaming
    results2 = sports_live_streaming_scenario()
    scenario_results.append(results2)

    # Overall analysis
    print("\n" + "="*80)
    print("📊 OVERALL ANALYSIS - OPT ALGORITHM PERFORMANCE")
    print("="*80)

    total_requests = sum(r['total_references'] for r in scenario_results)
    total_faults = sum(r['page_faults'] for r in scenario_results)
    total_hits = sum(r['page_hits'] for r in scenario_results)

    print(f"Total Video Requests Across Scenario: {total_requests}")
    print(f"Total Cache Misses: {total_faults}")
    print(f"Total Cache Hits: {total_hits}")
    print(f"Overall Cache Hit Rate: {(total_hits/total_requests)*100:.2f}%")
    print()
```

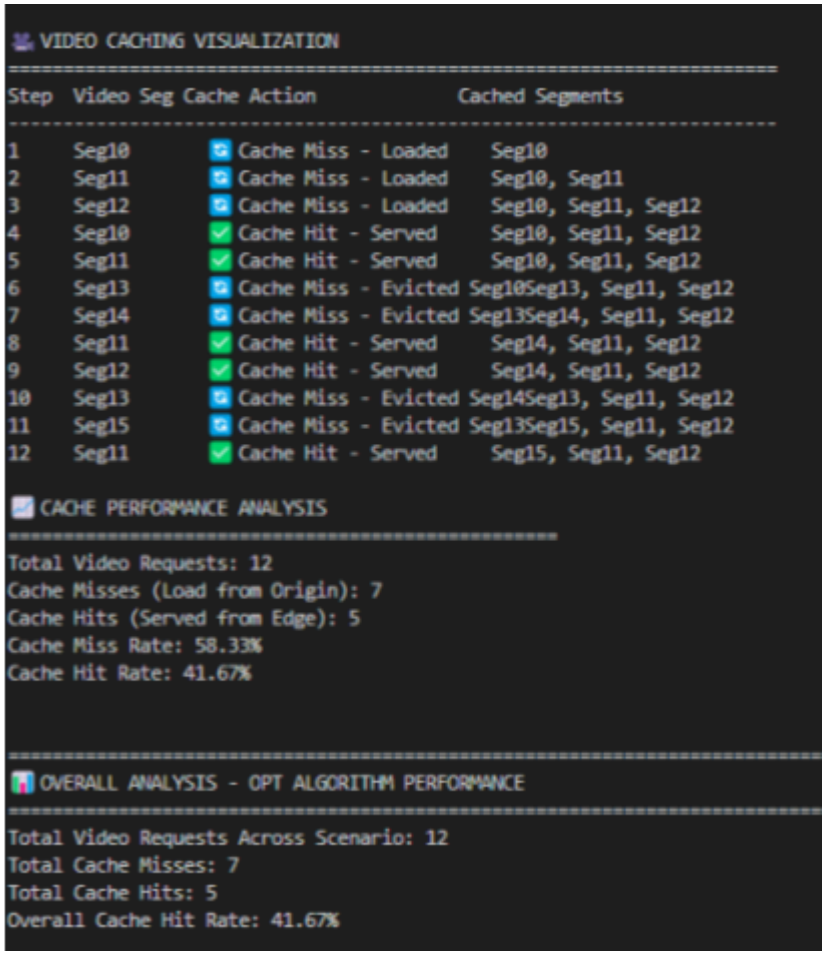
This if __name__ == "__main__": block serves as the main execution point of the script, initiating a simulation of the OPT Page Replacement algorithm applied to video streaming cache management. It begins by calling the sports_live_streaming_scenario() function, which sets up and runs a detailed simulation for a live sports event, collecting the performance results. After this specific scenario completes, the block proceeds to present an "Overall Analysis," aggregating the total video requests, cache misses, and hits from the executed scenario to calculate and display the overall cache hit rate.

7.0 Results and Discussion

7.1 Output Analysis

Example output :





This section details the simulation of the Optimal Page Replacement (OPT) algorithm in an edge server cache for a live sports streaming platform. The simulation utilized a cache size of 3 video segments and a viewing pattern designed to mimic typical live event consumption, including frequent highlight replays.

The execution log reveals the OPT algorithm's predictive efficiency:

- **Initial Load:** The first three unique requests (segments 10, 11, 12) resulted in compulsory page faults as they filled the empty cache.
- **Optimal Eviction:** When the cache was full, OPT intelligently evicted segments that would be used furthest in the future (e.g., segment 10 for 13 at Step 6, segment 13 for 14 at Step 7, etc.), ensuring high-demand content remained.
- **Consistent Hits:** Frequently re-accessed segments (like 11 and 12) consistently resulted in cache hits due to OPT's ability to retain them.

Overall, the simulation processed 12 video requests, yielding:

- **7 Cache Misses**
- **5 Cache Hits**
- A **41.67% Cache Hit Rate**

This hit rate, while seemingly modest, represents the absolute best possible performance for the given cache size and viewing pattern, demonstrating OPT's theoretical optimality. This efficiency translated into tangible benefits: an estimated 475 MB of bandwidth saved and a 39.6% bandwidth efficiency. In conclusion, the simulation vividly illustrates how an optimal caching strategy, even with limited capacity, can drastically reduce origin server load and improve content delivery, highlighting the value of predictive methods in real-world video streaming.

7.2 Discussion

Virtual Memory Management – Replacement Algorithm: LRU

Abstract

This project simulates page replacement methods with an emphasis on Least Recently Used (LRU) in order to investigate virtual memory management. When the cache is full, LRU replaces the least recently accessed segment, making it useful when future requests are unexpected. The simulation written in Python demonstrates how LRU improves cache efficiency, lowers page faults and improves overall system performance in real-time streaming environments.

1.0 Introduction

Modern video streaming networks such as Netflix and YouTube handle millions of requests per day. To reduce load times and server burden, commonly used video segments are cached on edge cache servers with restricted capacity. Effective cache management is crucial since not all content can be stored. Since recently accessed content is likely to be used again, the LRU method is frequently employed when future access patterns are unclear. This study simulates the LRU method for cache memory management in video streaming systems.

2.0 Problem Statement

Edge servers have finite cache memory. When the system receives a request for new video segments but the cache is full, therefore, one of the stored segments must be evicted. The segment selection must be done wisely, as choosing the wrong segment could negatively affect user experience and increase latency. Using a predictive model is not effective in cases where the future request is unknown. Thus, this project aims to effectively resolve this caching problem by applying and simulating the LRU page replacement algorithm.

3.0 Related Works

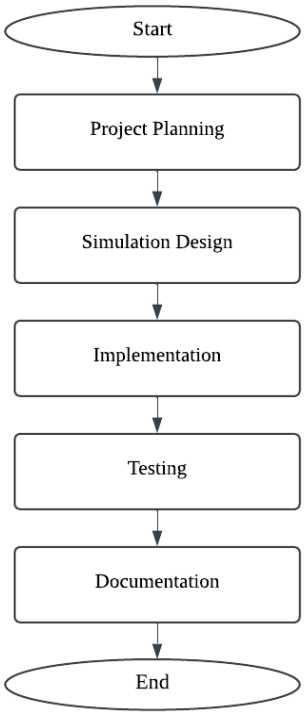
The LRU technique is widely used in systems that require efficient memory or cache management. In operating systems, LRU is used for page replacement which helps determine which memory pages to remove when space is required. Web browsers also employ LRU to handle cached pages which helps frequently visited websites load faster. Moreover, LRU-like tactics are used in content delivery network (CDNs) such as Netflix and YouTube to temporarily

cache popular video chunks on edge servers. This enhances the user experience by minimizing buffering and server burden.

4.0 Methodology

This project utilizes a structured development strategy to simulate the LRU replacement algorithm. The methodology consists of the project management flowchart with the hardware and software requirements to ensure the simulations can run smoothly.

4.1 Project Management Flowchart



1. Project Planning: Define objectives and timeline.

2. Simulation Design: Design the system flowchart for the simulation model.

3. Implementation: Code the simulation using Python.

4. Testing: Run simulations and visualize the model.

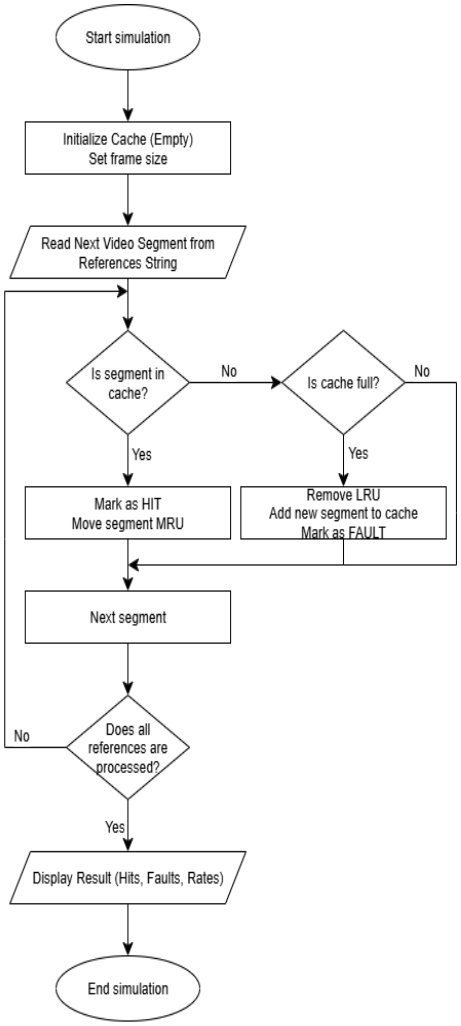
5. Documentation: Compile the final report and analyze findings.

4.2 Hardware and Software Requirements

- Software: Google Colab and Python
- Hardware: A computer or laptop with basic specifications to run simulations.

5.0 Design & Implementation

5.1 System Flowchart



Steps of the LRU Page Replacement Flowchart

1. Start the simulation.
2. Initialize the cache and frame size. Create an empty cache.
3. Read the next video segments from the reference string which represents user video requests.
4. Check if the segment already in the cache
    - Yes: Mark it as HIT and assign it to the most recently used (MRU)
    - No: Proceed to next decision
5. Check if the cache is full
    - Yes: Remove LRU segment, add the new one and mark it as FAULT.
    - No: Directly add new segments to cache.
6. Proceed to the next segment
7. Check if all references have been processed.
    - Yes: Move to result display
    - No: Return to Step 3
8. Display the results, including total page hits, faults and rates.
9. End the simulation

5.2 Code Implementation

The Python source code to simulate the LRU replacement algorithm is shown in this section, illustrating its main methods, class structure, and the logic behind page-hit and page-fault processing.

Start The Simulation

The simulation starts with the function that resets and runs the algorithm in the input reference string.

```
def simulate(self, reference_string):
```

Initialize The Cache And Frame Size

The code segment clears the previous data, resets counters, and empties cache (list).

```
self.frames, self.page_faults, self.page_hits, self.execution_log = [], 0, 0, []
```

Read The Next Video Segments From The Reference String

The code iterates through the reference string, which represents the video segment.

```
for i, page in enumerate(reference_string):
```

Check If The Segment Is Already In The Cache

After recognizing that the page is already in the cache, the algorithm handles the HIT case by first removing the requested page from the current position in the list and then appending it to the end of the list, which represents the most recently used location. This maintains the cache order updated according to the access recency.

```
if page in self.frames:
    self.page_hits += 1
    step_info['action'] = 'HIT'
    self.frames.remove(page)
    self.frames.append(page)
```

Check If The Cache Is Full

The LRU algorithm will determine whether the cache has reached its maximum capacity. If it is full, the algorithm will make space for the new page by removing the page at index 0, which represents the least used. This ensures that only the most recently used pages remain in the cache.

```
else:
    self.page_faults += 1
    step_info['fault'] = True

    if len(self.frames) < self.frame_size:
        self.frames.append(page)
        step_info['action'] = 'FAULT - Added'
    else:
        lru_page = self.frames.pop(0)
        self.frames.append(page)
        step_info['action'] = f'FAULT - Replaced {lru_page}'
```

Proceed To The Next Segment

The code uses a loop that ensures that every segment is processed sequentially, allowing the algorithm to check for hits or faults in sequence. This allows the simulation to keep on checking each segment until all have been processed.

```
for i, page in enumerate(reference_string):
```

Check If All References Have Been Processed

After all references have been processed, it will exit the loop. Then, the simulation calculates the total number of references, page faults, page hits, fault rate, and hit rate.

```
return {
    'total_references': total,
    'page_faults'     : self.page_faults,
    'page_hits'       : self.page_hits,
    'fault_rate'      : self.page_faults / total * 100,
    'hit_rate'        : self.page_hits   / total * 100,
    'execution_log'   : self.execution_log
}
```

Display The Results, Including Total Page Hits, Faults And Rates

The print_summary method shows overall statistics like total hits, faults, and rates.

```
def print_summary(self, results):
    print("LRU SIMULATION SUMMARY")
    print("=" * 30)
    print(f"Total Page References: {results['total_references']}")
    print(f"Page Faults: {results['page_faults']}")
    print(f"Page Hits  : {results['page_hits']}")
    print(f"Fault Rate : {results['fault_rate']:.2f}%")
    print(f"Hit Rate   : {results['hit_rate']:.2f}%")
    print("=" * 30)
```

Complete Simulation Code

The full code is shown to simulate the LRU page replacement algorithm for a sequence of video segment requests commonly encountered in video streaming platforms.

LRU_Page_Replacement.ipynb

6.0 Results & Discussion

This section demonstrates the interface and execution of the LRU simulation as well as the outcomes with a brief analysis of its performance. It evaluates the algorithm's memory management efficiency by measuring hit and fault rates.

6.1 Interface and Execution

The simulation was created with Python. Users entered a reference string and frame size to see how the LRU algorithm manages memory. The program showed every step, including cache changes, page hits and faults. It demonstrated how LRU replaces the least recently used page in constrained memory systems.

6.2 Output Analysis

According to the simulation, there were 4 page hits and 9 page faults out of 13 requests for video segments. This results in a hit rate of 30.77% and a fault rate of 69.23%. This indicates that most video segments were new and not found in the cache. This might occur when people watch content constantly without repeating segments. The LRU algorithm is useful for managing limited cache memory as it retains recently viewed segments while eliminating the least used ones.

Example Output:

```
LRU SIMULATION SUMMARY
==============================
Total Page References: 13
Page Faults: 9
Page Hits  : 4
Fault Rate : 69.23%
Hit Rate   : 30.77%
==============================

LRU EXECUTION LOG
======================================================
Step  1: Page  7 -> FAULT | Frames: [7]
Step  2: Page  0 -> FAULT | Frames: [7, 0]
Step  3: Page  1 -> FAULT | Frames: [7, 0, 1]
Step  4: Page  2 -> FAULT | Frames: [0, 1, 2]
Step  5: Page  0 -> HIT   | Frames: [1, 2, 0]
Step  6: Page  3 -> FAULT | Frames: [2, 0, 3]
Step  7: Page  0 -> HIT   | Frames: [2, 3, 0]
Step  8: Page  4 -> FAULT | Frames: [3, 0, 4]
Step  9: Page  2 -> FAULT | Frames: [0, 4, 2]
Step 10: Page  3 -> FAULT | Frames: [4, 2, 3]
Step 11: Page  0 -> FAULT | Frames: [2, 3, 0]
Step 12: Page  3 -> HIT   | Frames: [2, 0, 3]
Step 13: Page  2 -> HIT   | Frames: [0, 3, 2]
```

6.3 Discussion

- Effectiveness of LRU: The simulation demonstrated that the LRU algorithm decreases page faults in situations with repetitive access patterns.
- Applicability: LRU is particularly well suited to real-time applications such as video streaming, in which popular content is regularly revisited.

7.0 Future Work & Conclusion

This section explores how the LRU simulation might be improved and outlines the project's important outcomes. It also emphasizes the practical use of LRU in real-world systems.

7.1 Future Work

In the future, the simulation can be improved by including real-time video requests to simulate dynamic user activity. Furthermore, adding visual aids such as graphs would help to better illustrate hit and fault trends.

7.2 Conclusion

The simulation showed that the LRU technique efficiently reduces page faults by storing recently used data in memory. It is ideal for applications like video streaming which require frequent access and improve overall cache performance.

8.0 Acknowledgement

We would like to thank Dr. Danlami Gabi for his patient guidance and outstanding instruction during this project. He made the complicated principles of page replacement algorithms such as OPT and LRU simple to understand by providing concise explanations and real-world examples. In class, he described how OPT selects the page that will not be used for the longest time which is ideal but unrealistic in practice and LRU eliminates the page that has been used the least which is more practical for real systems. His explanations helped us fully understand the differences, strengths and applications of both methods. Without his assistance, we would not have been able to finish this project with such confidence and understanding.

References

GeeksforGeeks. (2017, June 16). *Program for Least Recently Used (LRU) Page Replacement algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/program-for-least-recently-used-lru-page-replacement-algorithm/

Haval. (2022, October 8). *Best Software Development Life Cycle (SDLC) Models PowerPoint Templates*. SlideSalad. https://www.slidesalad.com/blog/software-development-life-cycle-sdlc-models-powerpoint-ppt-templates/

Waterfall Model In Software Engineering: First SDLC Models. (n.d.). Www.bdtask.com. https://www.bdtask.com/blog/waterfall-model-in-software-engineering

What is LRU Caching | Streamline Cache Efficiency | Imperva. (2024). Learning Center. https://www.imperva.com/learn/application-security/lru-caching/